

Hypothesis testing

CIS 5450, Fall 2026

Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

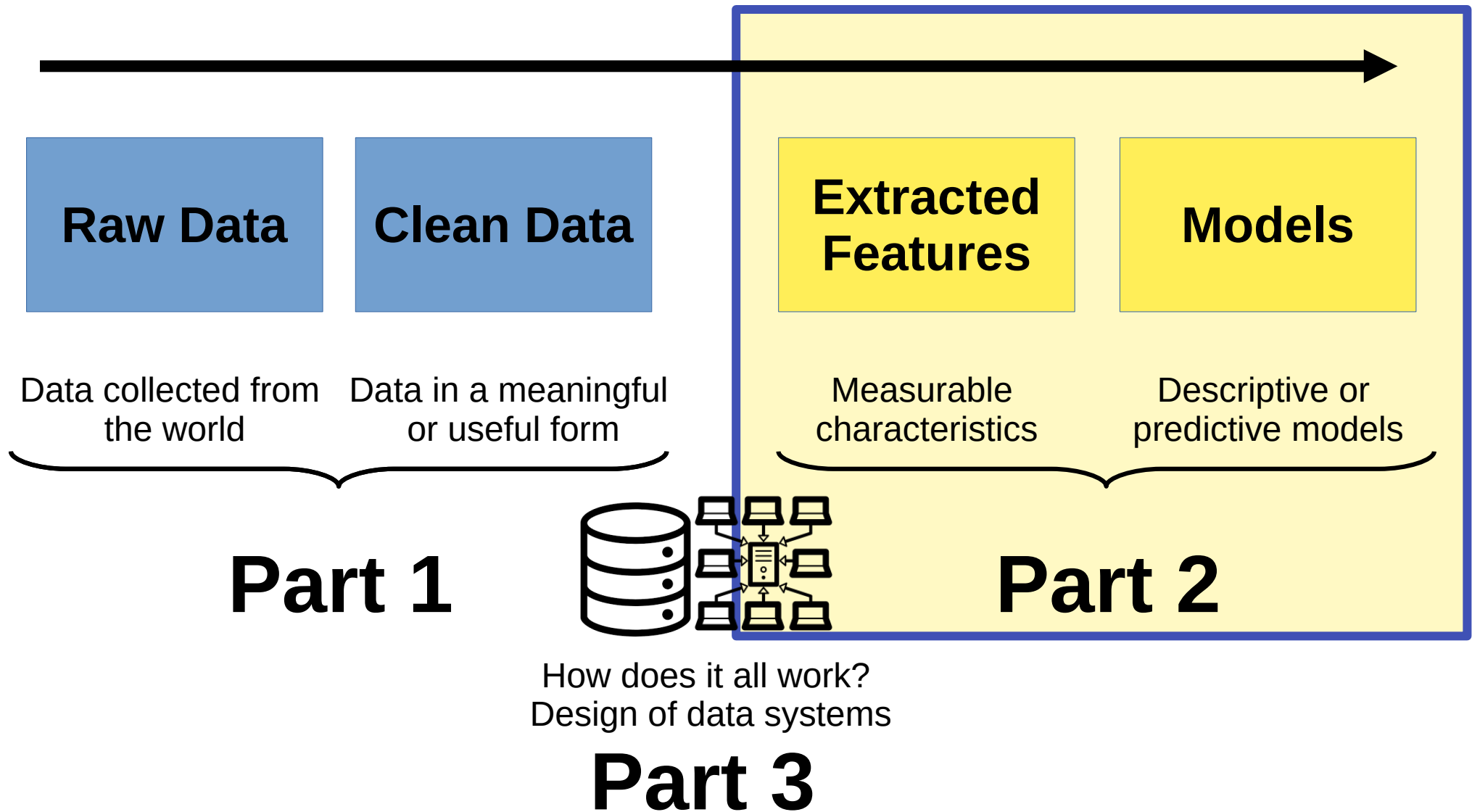
- **Course logistics**
- Brief review
- Hypothesis testing
- Vectorization

Upcoming Deadlines

- HW2 due Sep 30th at 10pm (2 days)
- HW3 releases at the same time
 - BUT, you will have until Oct 21st for HW3
- Midterm on Oct 7rd (9 days)
 - Practice exams online

Outline

- Course logistics
- **Brief review**
- Hypothesis testing
- Vectorization



Outline

- Course logistics
- Brief review
- **Hypothesis testing**
- Vectorization

Flipping a coin

- Suppose I flip a coin and get “heads” twice, and declare the coin rigged
- Suppose I flip a coin and get “heads” 100 times, and declare the coin fair

Is Yawning Contagious?

- Mythbusters episode: got random people into two rooms, had someone go yawn in one room, recorded how many yawners.

✓
0s

```
!wget -nc https://rmarcus.info/data/yawns.  
df = pd.read_csv("yawns.csv")  
df
```

↔ File 'yawns.csv' already there; not retrieving.

	person_id	group	measure
0	0	CONTROL	no yawn
1	1	CONTROL	no yawn
2	2	CONTROL	no yawn
3	3	CONTROL	no yawn
4	4	CONTROL	no yawn
5	5	CONTROL	no yawn
6	6	CONTROL	no yawn
7	7	CONTROL	no yawn
8	8	CONTROL	no yawn

CONTROL or TREAT

random people into
go yawn in one room,
others.

Effect: "no yawn" or
"yawn"

0s !wget -nc <https://rmarcus.info/data/yawns.csv>

16 in control group, 4
yawned

34 in treatment group,
10 yawned

person_id group measure

0 0 CONT

```
res = df.groupby(["group", "measure"]).count().reset_index()  
res = res.rename(columns={"person_id": "count"})  
res
```

4 4 CONT

5 5 CONT

6 6 CONT

7 7 CONT

8 8 CONT

group measure count

0 CONTROL no yawn 12

1 CONTROL yawn 4

2 TREAT no yawn 24

3 TREAT yawn 10

contagious?

random people into

no yawn in one room

✓
0s

```
!wget -nc https://rmarcus.info/data/yawns.csv  
df = pd.read_csv("yawns.csv")  
df
```

↗ File 'yawns.csv' already

✓
0s

	person_id	group
0	0	CONTROL
1	1	CONTROL
2	2	CONTROL
3	3	CONTROL
4		
5		
6		
7		
8		

25% of the control group
yawned

That's a 4pt difference!
Yawning is contagious!
... right?

```
[9] control_yawners = 4 / (12 + 4)  
    treat_yawners = 10 / (24 + 10)  
  
    control_yawners, treat_yawners
```

↗ (0.25, 0.29411764705882354)

29% of the treatment
group yawned

Example adapted from [Dr. Maria Tackett](#)

Contagious?

Is Yawning Contagious?

- Now, you may be remembering something from your stats class:

A t-test?

Assumes equal group sizes OR equal variances in each group

A χ^2 -test?

Not enough data! Table cell with < 5 entries.

A F-test?

Samples might not be drawn from a Gaussian population!

Is

- Now, you
your stats



Is?

Something from

hypothesis testing

A t-test?

al group sizes OR
es in each group

A χ^2 -test?

ata! Table cell with

A F-test?

nt not be drawn
ian population!



Is Yawning Contagious?

- We're going to be doing hypothesis testing via **simulation**
- Intuitively:
 - If we assume there is no effect, and we did a random trial, what is the probability of seeing a result like what we saw?

✓
0s

```
[9] control_yawners = 4 / (12 + 4)  
    treat_yawners = 10 / (24 + 10)
```

```
control_yawners, treat_yawners
```

⇒ (0.25, 0.29411764705882354)



```
total_yawners = (4 + 10) / (12 + 4 + 24 + 10)  
total_yawners
```



0.28

Hypothesis Testing

- Establish a **test statistic**: a measure of your effect.
- Set a **null hypothesis**: a model of the world in which your effect does not exist.
- **Simulate** the null world, plot the distribution of the test statistic

```
test_stat = treat_yawners - control_yawners  
test_stat
```

```
0.04411764705882354
```

“if yawning wasn’t contagious, this experiment just partitions people into rooms randomly (the treatment has no effect)”

Null world

- 16 people in the control room
- 34 people in the treatment room
- *If the null hypothesis is true*, then there's no difference between being in the control room and being in the treatment room
- So our samples, in the null world, are *exchangeable*

random seed helps make
our results reproducible

sting

Our exchangeable
population

- How to simulate
our null world?

Split the population
into two rooms

Compute null world
test stat

```
import random  
random.seed(42)
```

```
people = ("no yawn" * 12 + "yawn" * 4  
          + "no yawn" * 24 + "yawn" * 10)
```

```
random.shuffle(people)
```

```
c_room = people[0:16]
```

```
t_room = people[16:]
```

```
c_yawners = sum(x == "yawn" for x in c_room) / len(c_room)
```

```
t_yawners = sum(x == "yawn" for x in t_room) / len(t_room)
```

```
null_world_test_stat
```

```
null_world_test_stat
```



```
test_stat = treat_yawners - con  
test_stat
```

```
0.22794117647058826
```



```
0.04411764705882354
```

Simulate many iterations of the null world

```
import  
random.
```

```
all_test  
for _ in  
people
```

```
▶ _ = plt.hist(all_test_stats, bins=10)  
plt.xlabel("test statistic")  
plt.ylabel("# occurrences in null world")  
plt.grid(axis="y", ls="--")
```

How often did we see an effect this strong or stronger in the null world?

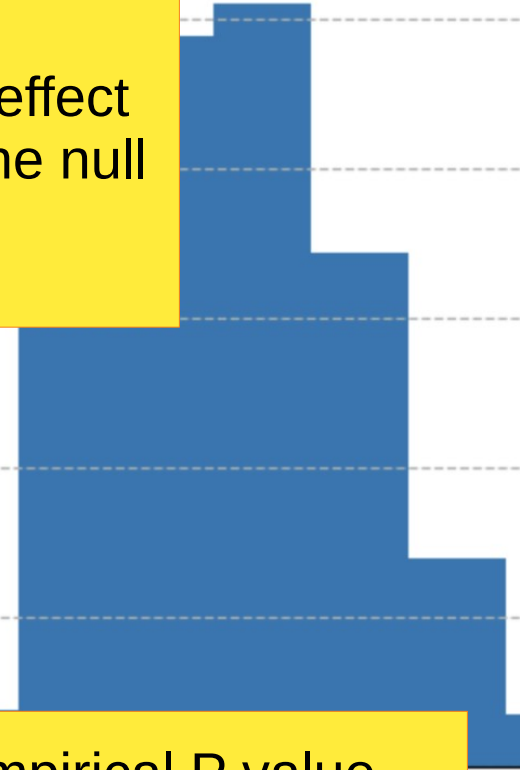
```
p_value = (sum(x >= test_stat for x in all_test_stats)  
           / len(all_test_stats))
```

```
p_value
```

```
0.513559
```

The empirical P value

test statistic



Hypothesis Testing

- Our null hypothesis assumed there was no difference between the treatment and control rooms
 - ... so therefore if we scramble people between rooms, we should see the same result!
- *Permutation testing*

Hypothesis Testing

- P values can be confusing.

Assuming the null hypothesis (treat & control are the same), there is a $p=51\%$ chance of observing an equal or stronger effect

Assuming null, If you did this experiment 1000 times, you would expect 510 runs to show an equal or stronger effect

There is only a 49% chance the null hypothesis is true!

FALSE

There is a $100 - p = 49\%$ chance that yawning is contagious!

FALSE

Hypothesis Testing

- If yawning is not contagious, but random, there is a $p=51\%$ chance of seeing this result.
- We normally require $p < 0.05$ or 0.01 to “reject” a null hypothesis.
- So we cannot rule out the possibility that the results are explained by chance

✓
0s

```
[9] control_yawners = 4 / (12 + 4)
    treat_yawners = 10 / (24 + 10)

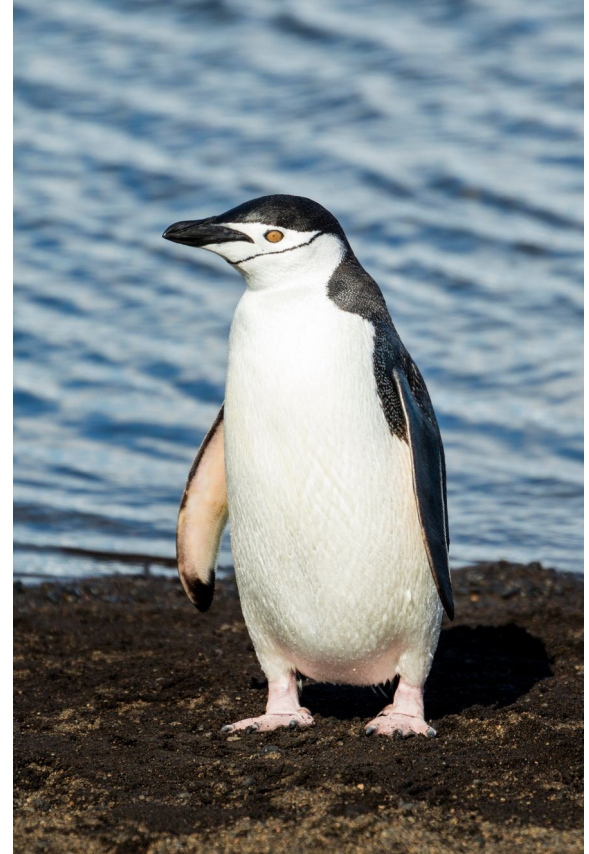
    control_yawners, treat_yawners
```



```
(0.25, 0.29411764705882354)
```

So, is yawning contagious?

Penguins



Do Gentoo penguins weigh more than Chinstrap penguins?

Penguins

Dataset has 3 different types of penguins

We will just get the Chinstraps and Gentoos

About 1kg difference on average

```
!wget -nc https://rmarcus.info/data/penguins.csv  
df = pd.read_csv("penguins.csv")  
df["species"].value_counts()
```

File 'penguins.csv' already there; not retrieving.

	count
species	
Adelie	152
Gentoo	124
Chinstrap	68

dtype: int64

```
chinstraps = df[df["species"] == "Chinstrap"]  
gentoos = df[df["species"] == "Gentoo"]  
  
gentoos["body_mass_g"].mean() - chinstraps["body_mass_g"].mean()
```

1342.928024868484

Test stat: diff
of means



```
df = df.dropna()
chinstraps = df[df["species"] == "Chinstrap"]
gentoos = df[df["species"] == "Gentoo"]
gentoos["body_mass_g"].mean() - chinstraps["body_mass_g"].mean()

np.float64(1357.5367647058824)
```

Simulate the null
world

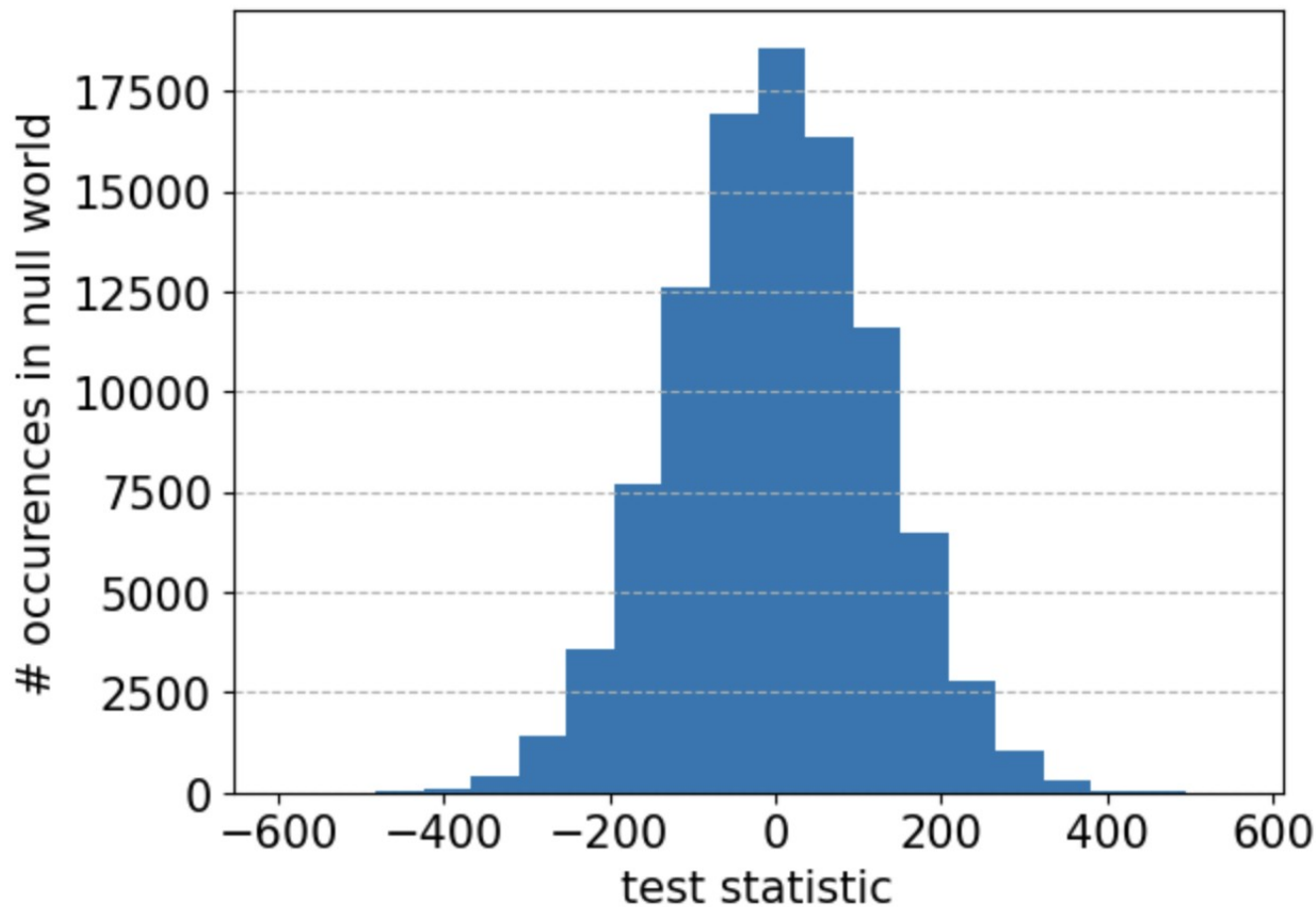


```
all_penguins = (chinstraps["body_mass_g"].to_list()
                + gentoos["body_mass_g"].to_list())

all_test_stats = []
for _ in range(1_000_000):
    random.shuffle(all_penguins)
    null_chinstraps = all_penguins[:len(chinstraps)]
    null_gentoos = all_penguins[len(chinstraps):]
    diff = np.mean(null_gentoos) - np.mean(null_chinstraps)
    all_test_stats.append(diff)
```

```
plt.hist(all_test_stats, bins=20)  
plt.xlabel("test statistic")  
plt.ylabel("# occurrences in null world")  
plt.grid(axis="y", ls="--")
```

Empirical $P < 1/100000$



Penguins

- We can **reject the null hypothesis!**
- Assuming the null hypothesis, there is less than a one in a million chance of observing our data.
- Intuitively, the observed differences are attributable to **more than just chance.**

State GDP Per Capita

- GDP/c is a common metric of economic well-being
 - California: 102k. Missouri: 70k
- Common hypothesis: coastal cities have ports and therefore have higher GDP/c

```
gdp_data = pd.read_html("https://rmarcus.info/data/gdpc.html")[0]
gdp_per_cap = gdp_data.iloc[:, [0, 7]]
gdp_per_cap.columns = ["state", "gdp_per_cap"]
gdp_per_cap = gdp_per_cap.drop(51) # last row is US average
gdp_per_cap
```

...	state	gdp_per_cap	
0	California	\$104,916	
1	Texas	\$86,987	
2	New York	\$117,332	
3	Florida	\$73,784	
4	Illinois	\$90,449	
5	Pennsylvania	\$78,544	
6	Ohio	\$78,120	
7	Georgia	\$78,754	
8	Washington	\$108,468	
9	New Jersey	\$90,272	

State GDP Per Capita

- Integrate with a list of coastal states, clean up the data

```
coastal_states = ["Alaska", "Florida", "California", "Hawaii",  
                 "Louisiana", "Texas", "North Carolina",  
                 "Texas", "North Carolina", "Oregon",  
                 "Maine", "Massachusetts", "South Carolina",  
                 "Washington", "New Jersey", "New York",  
                 "Virginia", "Georgia (U.S. state)", "Connecticut",  
                 "Alabama", "Mississippi", "Rhode Island",  
                 "Maryland", "Delaware", "New Hampshire"]  
  
gdp_per_cap["coastal"] = gdp_per_cap["state"].isin(coastal_states)  
gdp_per_cap["gdp_per_cap"] = (  
    gdp_per_cap["gdp_per_cap"]  
    .str.replace("$,", "", regex=True)  
    .astype(int)  
)  
gdp_per_cap
```

```
...      state  gdp_per_cap  coastal  
0      California      104916      True  
1           Texas       86987      True  
2       New York      117332      True  
3         Florida       73784      True  
4        Illinois       90449     False
```



State GDP Per Capita

```
avg_coastal = gdp_per_cap[gdp_per_cap["coastal"]]["gdp_per_cap"].mean()  
avg_non_coastal = gdp_per_cap[~gdp_per_cap["coastal"]]["gdp_per_cap"].mean()  
avg_coastal - avg_non_coastal
```

```
np.float64(1403.9809523809527)
```



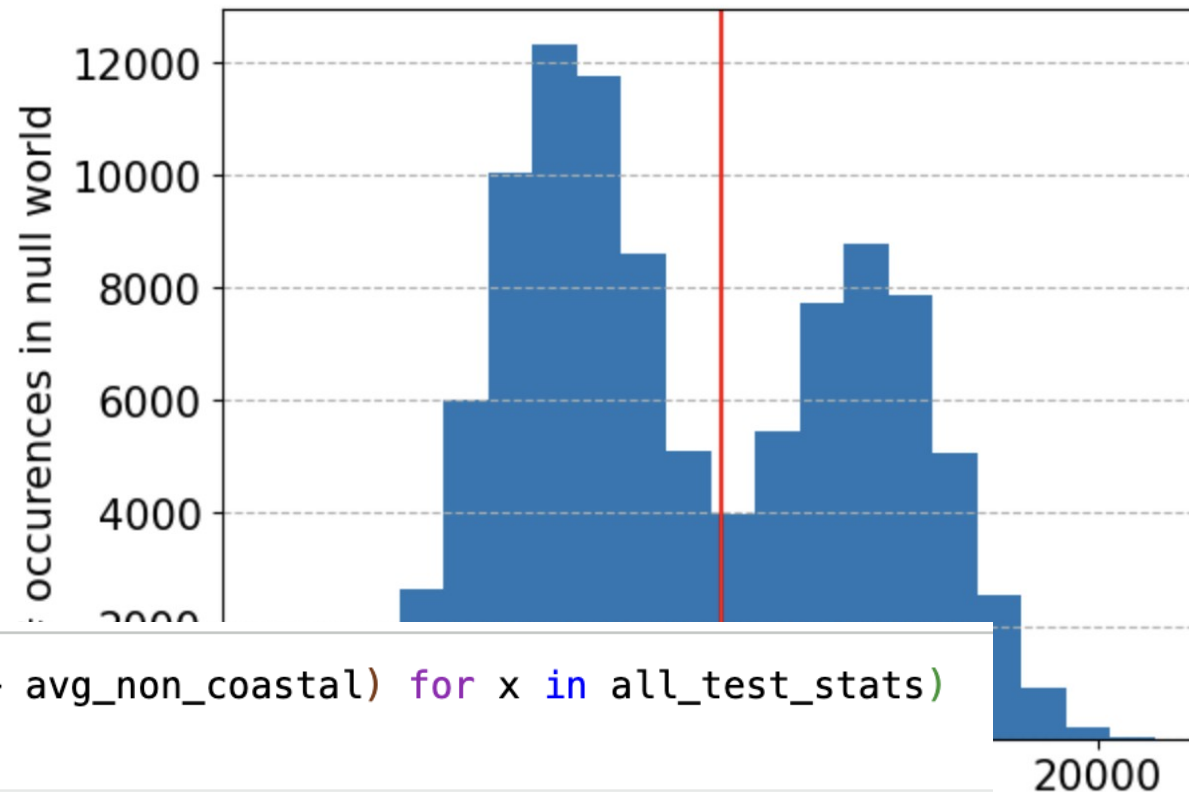
Test statistic

```
num_coastal = len(
num_non_coastal =
gdps = gdp_per_cap
```

```
all_test_stats = []
for _ in range(100):
    random.shuffle(gdps)
    null_avg_coastal =
    null_avg_non_coastal =
    all_test_stats.append(
```

```
plt.hist(all_test_stats, bins=20)
plt.xlabel("Test statistic")
plt.ylabel("# occurrences in null world")
plt.grid(axis="y", ls="--")
plt.axvline(avg_coastal - avg_non_coastal, color="red")
```

```
>>> <matplotlib.lines.Line2D at 0x7b3192c52bd0>
```



```
(sum(x > (avg_coastal - avg_non_coastal) for x in all_test_stats)
 / len(all_test_stats))
```

```
np.float64(0.4163)
```

The Downside: Time

```
%%time
num_coastal = len(gdp_per_cap[gdp_pe
num_non_coastal = len(gdp_per_cap[~g
gdps = gdp_per_cap["gdp_per_cap"].to
```

```
all_test_stats = []
for _ in range(100_000):
    random.shuffle(gdps)
    null_avg_coastal = np.mean(gdps[:n
    null_avg_non_coastal = np.mean(gdp
    all_test_stats.append(null_avg_coa
```

```
CPU times: user 2.42 s, sys: 3.63 ms
Wall time: 3.65 s
```

```
%%time
all_penguins = (chinstraps["body_mass_g"].to_list()
                + gentoos["body_mass_g"].to_list())
```

```
all_test_stats = []
for _ in range(100_000):
    random.shuffle(all_penguins)
    null_chinstraps = all_penguins[:len(chinstraps)]
    null_gentoos = all_penguins[len(chinstraps):]
    diff = np.mean(null_gentoos) - np.mean(null_chinstraps)
    all_test_stats.append(diff)
```

```
CPU times: user 6.88 s, sys: 10.5 ms, total: 6.89 s
Wall time: 8.1 s
```

Outline

- Course logistics
- Brief review
- Hypothesis testing
- **Vectorization**

Vectorization

- Python, as a programming language, often trades speed for ease-of-use.

Vectorization

- Python, as a programming language, often trades speed for ease-of-use.
- ... but we've seen that doing thing with Python libraries can be fast!
 - DuckDB, Polars
 - These libraries do all the hard work for you, and give you a nice interface
- Another similar tool: NumPy (numeric python)



Vectorization

- Numpy operates in a *vectorized* or *SIMD* fashion
 - SIMD: Single Instruction, Multiple Data
- Allows you to apply the same operation to many numbers very quickly

Vectorization

✓
0s

```
[18] import numpy as np  
     arr = np.arange(0, 100_000)  
     arr
```

Create an array with integers from 0 to 100k

⇒ array([0, 1, 2, ..., 99997,

✓
0s

```
[20] arr + 2
```

Add a scalar (2) to the array, *broadcasts* to the whole array

⇒ array([2, 3, 4, ..., 99999, 100000, 100001])

Vectorization

✓
0s

```
▶ a = np.arange(0, 5)  
  b = np.arange(15, 20)  
  
  a + b
```

```
⇒ array([15, 17, 19, 21, 23])
```

Add two arrays together
element-by-element

Vectorization

- Vectorization requires different approaches.
- If I sample the unit Gaussian, 1M times, how many times will a get

Generate 1M unit Gaussian samples

Broadcast comparison, gives an array of booleans

True is 1, False is 0, so the sum is the count

```
counter = 0
for i in range(1000000):
    random.gauss(0, 1)
    counter += 1

counter
```

CPU times: user 1.22 s, sys: 4.98 ms, total: 1.23 s
Wall time: 1.32 s
1391

```
%time
np.sum(np.random.normal(0, 1, 1_000_000) > 3)
```

CPU times: user 34.6 ms, sys: 0 ns, total: 34.6 ms
Wall time: 34 ms
1348

Vectorizing Penguins

Special numpy rng, which supports vectorization

Array of 0, 1, 2, ...

100k rows, each row is 0, 1, 2, ...

Each row still has 0...n, but in random order

Build samples and compute mean all at once

```
%%time
rng = np.random.default_rng(seed=42)
masses = np.array(chinstraps["body_mass_g"].to_list()
                  + gentoos["body_mass_g"].to_list())

indexes = np.arange(0, len(all_penguins))
indexes = np.tile(indexes, (100_000, 1))
indexes = rng.permuted(indexes, axis=1)
samples = masses[indexes]
c_means = np.mean(samples[:,0:len(chinstraps)], axis=1)
g_means = np.mean(samples[:,len(chinstraps):], axis=1)
all_test_stats = g_means - c_means
```

CPU times: user 475 ms, sys: 73.2 ms, total: 548 ms
Wall time: 800 ms

Vectoriz

```
%%time
rng = np.random.default_rng(seed=42)
masses = np.array(chinstraps["body_mass_g"]
                  + gentoos["body_mass_g"]
```

```
indexes = np.arange(0, len(all_penguins)
indexes = np.tile(indexes, (100_000, 1)).
indexes = rng.permuted(indexes, axis=1)
samples = masses[indexes]
c_means = np.mean(samples[:,0:len(chinstraps)], axis=1)
g_means = np.mean(samples[:,len(chinstraps):], axis=1)
all_test_stats = g_means - c_means
```

CPU times: user 475 ms, sys: 73.2 ms, total: 548 ms
Wall time: 800 ms

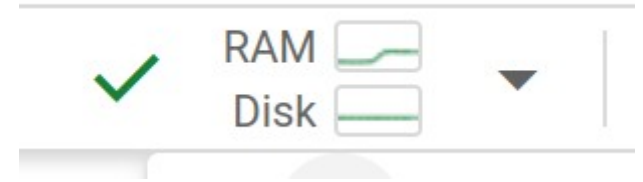
```
%%time
all_penguins = (chinstraps["body_mass_g"].to_list()
               + gentoos["body_mass_g"].to_list())

all_test_stats = []
for _ in range(100_000):
    random.shuffle(all_penguins)
    null_chinstraps = all_penguins[:len(chinstraps)]
    null_gentoos = all_penguins[len(chinstraps):]
    diff = np.mean(null_gentoos) - np.mean(null_chinstraps)
    all_test_stats.append(diff)
```

CPU times: user 6.88 s, sys: 10.5 ms, total: 6.89 s
Wall time: 8.1 s

Vectorization: The Downside

- Downside to vectorization: memory usage
- Large arrays requires space
- Fix: batching!



What about other variables?

- Tests today are generally of a single binary variable: treatment / control.
- What about when there is more than one variable?
 - Ports, policies, venture funding, tax rates → GDP?
- Next time: regression analysis